SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	*Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.*		
Student Name:	Nivuthika Sai Prakash	Reg. No.:	192511294
Section / Batch:	2nd year / CSE	Date:	04/09/2026

Code of Conduct

I Nivuthika Sai Prakash (192511294) *(Name / Reg No)* *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Nivuthika Sai Prakash

Instructions

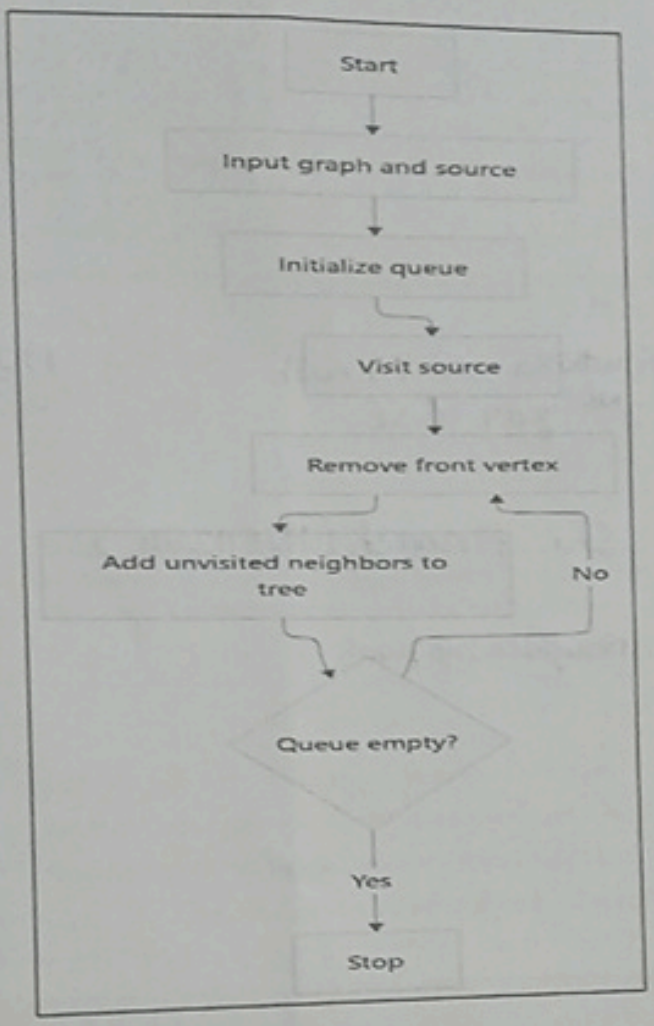
- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Shortest Path Algorithm	Dijkstra's Algorithm	2	20
Shortest Path Algorithm	Unweighted Graph	3	20
Graph Traversal	BFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
		GRAND TOTAL:	**100 Marks**

Annexure A – Pseudocode Writing Task

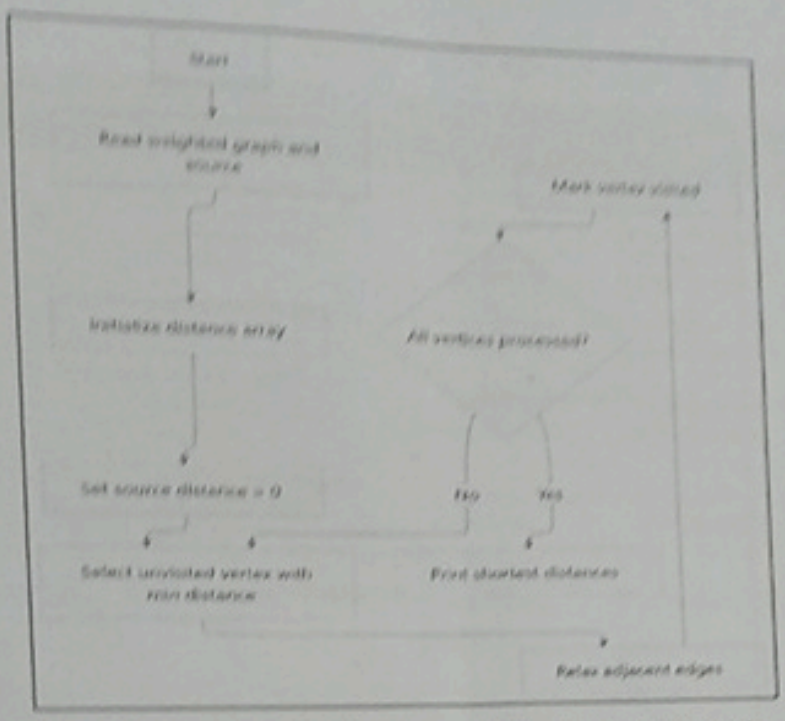
1. Convert the flowchart for generating a BFS tree into code. (20 Marks)

Flowchart Diagram:



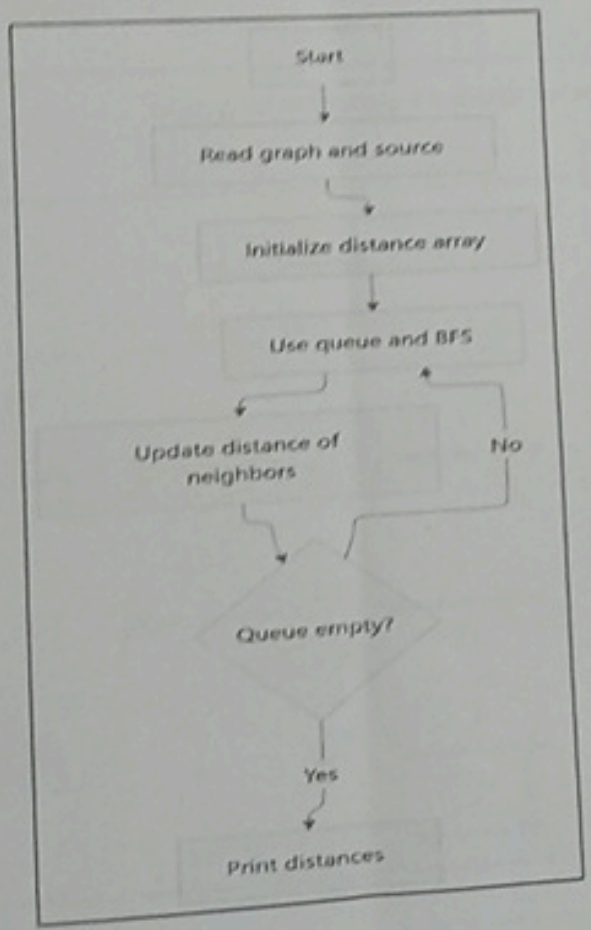
2. Convert the flowchart for Dijkstra's shortest path algorithm into code. (20 Marks)

Flowchart Diagram:



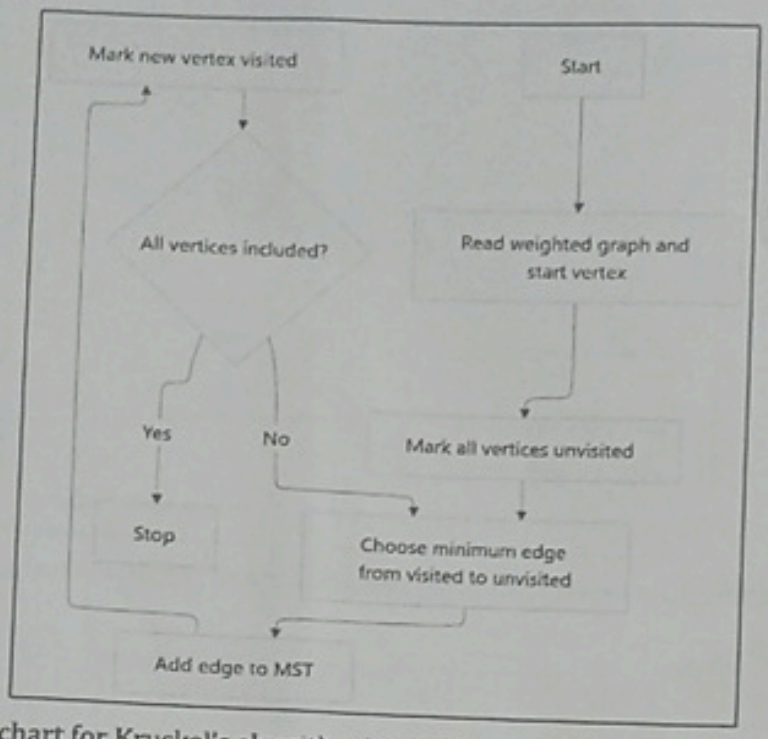
3. Convert the flowchart for shortest path in an unweighted graph into code. (20 Marks)

Flowchart Diagram:



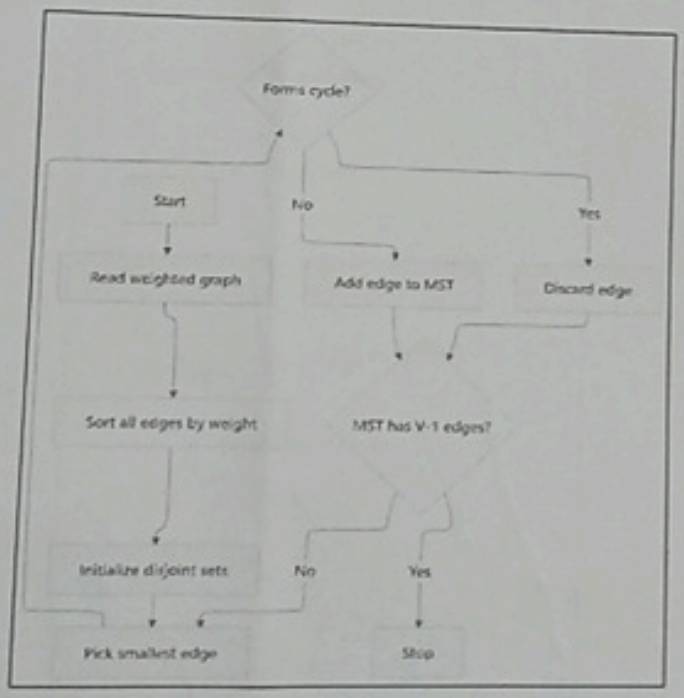
4. Convert the flowchart for Prim's algorithm into code. (20 Marks)

Flowchart Diagram:



5. Convert the flowchart for Kruskal's algorithm into code. (20 Marks)

Flowchart Diagram:



Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature:	Signature:	Signature:
Date:	Date:	Date:

CO5 - Assessment Tool 2

Name : Nirmithika Sri Prakash

Reg no : 192511294

Course code : CSA0305

Course : Data Structures

(1)

BFS - Breadth First Search

```
#include <stdio.h>
int main() {
    int n, graph[20][20], visited[20] = {0};
    int queue[20], front = 0, rear = 0;
    int start, i, v;
    Printf ("Enter number of vertices: ");
    Scanf ("%d", &n);
    Printf ("Enter adjacency matrix: \n");
    for (i = 0; i < n; i++)
        for (v = 0; v < n; v++)
            Scanf ("%d", &graph[i][v]);
    Printf ("Enter starting vertex: ");
    Scanf ("%d", &start);
    queue [rear++] = start;
    visited [start] = 1;
    Printf ("BFS Traversal: ");
    While (front < rear) {
        v = queue
```

```
printf("%d", v);
for(i=0; i<n; i++){
    if(graph[v][i]==1 && visited[i]==
    { visited[i]=1
      queue[rear++]=i;
    }
}
return 0;
}
```

(2) Dijkstra's Shortest Path Algorithm

```
#include<stdio.h>
int main(){
int n, graph[20][20];
int dist[20], visited[20]={0};
int i,j, count, u, min, start;
Printf("Enter number of vertices: ");
Scanf("%d", &n);
Printf("Enter weighted adjacency: \n");
for(i=0; i<n; i++)
  for(j=0; j<n; j++)
Scanf("%d", &graph[i][j]);
Printf("Enter source vertex: ");
Scanf("%d", &start);
for(i=0; i<n; j++)
dist[i] = graph[start][i];
dist[start]=0;
visited[start]=1;
```

```
min = INF;
u = -1;
for (i=0; i<n; i++){
if (!visited[i] && dist[i] < min){
    min = dist[i];
    u = i;
}
}
if (u == -1)
break;
visited[u] = 1;
for (i=0; i<n; i++){
if (!visited[i] && graph[u][i] != 0 &&
    dist[u] + graph[u][i] < dist[i]){
    dist[i] = dist[u] + graph[u][i];
}
}
}
printf("Shortest distances: \n");
for (i=0; i<n; i++)
printf("%d -> %d = %d \n", start, i, dist[i]);
return 0;
}
```

(3) Shortest Path in an unweighted graph

```
#include <stdio.h>
int main(){
int n, graph[20][20];
int dist[20], visited[20] = {0};
int queue[20], front=0 rear=0;
```

```
    scanf("%d", &n);
    printf("Enter adjacency matrix: ");
    for(i=0; i<n; i++)
        for(v=0; v<n; v++)
            scanf("%d", &graph[i][v]);
    for(i=0; i<n; i++)
        dist[i] = -1;
    queue[rear++] = start;
    visited[start] = 1;
    dist[start] = 0;
    for(i=0; i<n; i++) {
        if(graph[v][i] == 1 && !visited[i]) {
            visited[i] = 1;
            dist[i] = dist[v] + 1;
            queue[rear++] = i;
        }
    }
}
printf("Shortest distances from %d:\n", start);
    return 0;
}
```

---

(4) Prim's Minimum Spanning Tree

=

```
#include <stdio.h>
int main() {
    int n, graph[20][20];
    int selected[20] = {0};
    int i, j, edges=0;
```

```
Printf("Enter number of Vertices: ");
Scanf("%d", &n);
Printf("Enter weighted adjacency matrix: \n");
for(i=0; i<n; i++)
  for(j=0; j<n; j++)
  Scanf("%d", &graph[i][j]);
    Select[0] = 1;
    for(i=0; i<n; i++){
      if(selected[i]){
      for(j=0; j<n; j++){
       if(!selected[j] && graph[i][j]!=0
          && graph[i][j]<min){
          min = graph[i][j];
              x=i;
              y=j;
          }
      }
    if(x==-1)
    break;
  Printf("minimum cost = %d \n", total);
    return 0;
  }
```

(5) Kruskal's Minimum Spanning Tree

```
#include<stdio.h>
  int parent[20];
  int find(int x){
    if(parent[x]==x)
      return x;
```

```
    return Parent[x] = find(Parent[x]);
}
void unionset(int a, int b) {
    Parent[find(a)] = find b;
}
int main() {
    int n, e, i, j;
    int count = 0, total = 0;
    Printf("enter number of vertices: ");
    Scanf("%d", &n);
    Printf("Enter number of edges: ");
    Scanf("%d", &e);
    for(i=0; i<e; i++)
        Scanf("%d %d %d", &edge[i].u, &edge[i].v);
    for(i=0; i<n; i++)
        Parent[i] = i;
    for(i=0; i<e-1; i++) {
        for(j=0; j<e-1-1; j++) {
            if(edge[j].weight > edge[j+1].weight)
            { temp = edge[j];
                edge[j] = edge[j+1];
                edge[j+1] = temp;
            }
        }
    }
    Printf("Minimum Cost = %d\n", total);
    return 0;
}
```